Chapter 13 — Serving & Deployment

13.1 Inference Engine Internals: Batching, PagedAttention, vLLM

Conceptual Deep Dive

Decoding is memory-bandwidth-bound, so serving one request at a time wastes the GPU. **Static batching** waits for a full batch and is gated by the longest sequence. **Continuous batching** (a.k.a. in-flight batching) schedules at the *iteration* level: when a sequence finishes, a waiting request takes its slot immediately — large throughput gains. **PagedAttention** (vLLM) stores the KV cache in fixed-size blocks mapped by a page table, eliminating fragmentation (waste drops from ~60–80% to <5%) and enabling **prefix caching** and copy-on-write sharing for beam search / shared system prompts. Other optimisations: **chunked prefill** (interleave prompt processing with decoding to protect latency), **speculative decoding** (a small draft model proposes k tokens; the large model verifies them in one pass; output distribution unchanged), **FlashAttention**, tensor/pipeline parallelism, quantised weights/KV cache, **disaggregated prefill/decode**, and LoRA multi-adapter batching. Engines: **vLLM**, **SGLang** (RadixAttention prefix tree, fast structured output), **TensorRT-LLM**, **TGI**, LMDeploy.

Cross-Connection Scenarios

KV cache sizing (5.4); structured output via XGrammar (7.3); quantisation formats (12.5); latency metrics (10.6).

Production Anti-Patterns

- `max_model_len` set far above need, shrinking batch capacity.
- Benchmarking with synthetic equal-length prompts.
- Running an engine at 100% `gpu_memory_utilization` leaving no headroom for spikes.

Minimal Working Code

```
# pip install vllm   (Linux + NVIDIA GPU)
from vllm import LLM, SamplingParams

llm = LLM(model="Qwen/Qwen2.5-7B-Instruct", dtype="bfloat16",
          gpu_memory_utilization=0.90, max_model_len=8192,
          enable_prefix_caching=True)
params = SamplingParams(temperature=0.2, max_tokens=256)
outs = llm.chat([[{"role": "user", "content": "Explain PagedAttention
in 2 sentences."}]], params)
print(outs[0].outputs[0].text)

# Server (OpenAI-compatible):
#   vllm serve Qwen/Qwen2.5-7B-Instruct --max-model-len 8192 --enable-prefix-caching --port 8000
```

13.2 Alternative Servers: TGI, Triton, Ollama, llama.cpp

Conceptual Deep Dive

Tool	Best for	Notes
vLLM	General high-throughput GPU serving	OpenAI-compatible API, broad model support
SGLang	Prefix-heavy, structured-output, agent workloads	RadixAttention, fast JSON
TGI (Hugging Face)	HF-ecosystem production	Mature, Rust router, now in maintenance-leaning mode — check status
TensorRT-LLM + Triton Inference Server	Max NVIDIA performance, multi-model/ensemble pipelines	Engine compile step; Triton gives dynamic batching, model repository, metrics, gRPC
Ollama	Local dev, laptops, small teams	Wraps llama.cpp; Modelfile; GGUF; simple REST
llama.cpp	CPU/Apple-silicon/edge	GGUF, quantised, low footprint
Managed alternatives: Bedrock, Vertex AI, Azure AI Foundry, Together, Fireworks, Groq, Modal, Replicate, Baseten. Decide using build-vs-buy: at low/spiky volume APIs win on cost; at steady high volume with privacy or latency needs, self-host.		

Cross-Connection Scenarios

Cost modelling (2.4); security/data residency (14); GPU provisioning (13.5).

Production Anti-Patterns

- Using Ollama as a high-concurrency production server.
- Self-hosting for a workload of 1k requests/day.
- Vendor-specific API shapes in application code (no abstraction layer).

Minimal Working Code

```
# Ollama: `ollama pull llama3.2` then call the local server
import httpx

def ollama_chat(prompt: str, model="llama3.2") -> str:
    r = httpx.post("http://localhost:11434/api/chat", timeout=120,
                   json={"model": model, "stream": False,
                         "messages": [{"role": "user", "content":
 prompt}]})
    r.raise_for_status()
    return r.json()["message"]["content"]

# Any OpenAI-compatible server (vLLM/SGLang/TGI/Ollama /v1) with one client:
#   from openai import OpenAI; c = OpenAI(base_url="http://localhost:8000/v1", api_key="x&
quot;)
```

13.3 Containerisation with Docker

Conceptual Deep Dive

Containers give reproducible environments. For GPUs install the **NVIDIA Container Toolkit** and run with `--gpus all`. Practices: slim/multi-stage builds, pin base image digest and dependency versions, run as non-root, don't bake large model weights into images unless needed — mount a volume or pull from object storage at start with a cache, add health (/health) and readiness

probes, set resource limits, handle SIGTERM for graceful shutdown (finish in-flight streams), and scan images for CVEs.

Cross-Connection Scenarios

CI builds images (15); Kubernetes runs them (13.4); secrets management (14).

Production Anti-Patterns

- 25 GB images rebuilt on every code change.
- Secrets in ENV of the image.
- latest tags.

Minimal Working Code

```
# Dockerfile
FROM vllm/vllm-openai:v0.6.3
ENV HF_HOME=/models
RUN useradd -m app
USER app
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --start-period=300s CMD curl -f http://localhost:8000/health || e
xit 1
ENTRYPOINT ["python3", "-m", "vllm.entrypoints.openai.api_server", \
          "--model", "Qwen/Qwen2.5-7B-Instruct", "--max-model-len", "
;8192"]
```

```
# graceful shutdown for a custom FastAPI worker
import signal
import asyncio

shutdown = asyncio.Event()

def handle_sigterm(*_):
    shutdown.set()

signal.signal(signal.SIGTERM, handle_sigterm)
```

13.4 Kubernetes and Autoscaling

Conceptual Deep Dive

Run inference as a Deployment requesting nvidia.com/gpu, using the NVIDIA device plugin/GPU Operator, node selectors/taints for GPU pools, and PVCs or init-containers for weights. Standard CPU-based HPA is a poor signal for LLMs; autoscale on **queue depth, running requests, or GPU KV-cache utilisation** (exported via Prometheus) using **KEDA** or HPA with custom metrics. Cold starts are minutes (image pull + weight load): keep warm replicas, use image/weight caching, and scale on leading indicators. Frameworks: **KServe**, **Ray Serve**, **KubeRay**, **llm-d**, Knative. Use PodDisruptionBudgets, rolling updates with maxUnavailable: 0, and canary traffic splitting (Argo Rollouts/Istio). Multi-GPU models need tensor-parallel-size equal to GPUs on a node with NVLink.

Cross-Connection Scenarios

Metrics (14); canary deployments (15); cost control (2.4).

Production Anti-Patterns

- Scaling to zero for latency-sensitive interactive endpoints.
- Autoscaling on CPU.
- No pod anti-affinity → all replicas on one node.

Minimal Working Code

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: llm }
spec:
  replicas: 2
  strategy: { rollingUpdate: { maxUnavailable: 0, maxSurge: 1 } }
  selector: { matchLabels: { app: llm } }
  template:
    metadata: { labels: { app: llm } }
    spec:
      terminationGracePeriodSeconds: 120
      containers:
        - name: vllm
          image: vllm/vllm-openai:v0.6.3
          args: ["--model", "Qwen/Qwen2.5-7B-Instruct", "--max-model-len",
"8192"]
          resources: { limits: { nvidia.com/gpu: 1 } }
          readinessProbe: { httpGet: { path: /health, port: 8000 }, initialDelaySeconds: 120 }
          ports: [{ containerPort: 8000 }]
---
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: { name: llm-scaler }
spec:
  scaleTargetRef: { name: llm }
  minReplicaCount: 2
  maxReplicaCount: 8
  triggers:
    - type: prometheus
      metadata:
        serverAddress: http://prometheus:9090
        query: avg(vllm:num_requests_waiting)
        threshold: "5"
```

13.5 Cloud GPU Deployment: AWS, Lambda Labs and Beyond

Conceptual Deep Dive

GPU choice by workload: **inference of 7–13B** → L4/A10G/L40S; **70B** → 2–4×A100/H100 or quantised on 1×80 GB; **training/fine-tuning** → A100/H100/H200/B200 with NVLink. AWS: EC2 g5/g6/g6e (A10G/L4/L40S), p4d/p5 (A100/H100), SageMaker endpoints, Bedrock (managed models + custom import), EKS with GPU node groups, Spot instances for batch (checkpoint to survive interruption), Savings Plans for steady load. **Lambda Labs, RunPod, CoreWeave, Vast.ai** offer cheaper on-demand/reserved GPUs with less managed tooling; **Modal/Baseten** give serverless GPUs with fast cold starts. Weigh price/hour, availability, egress fees, network (EFA/InfiniBand), compliance (SOC2, HIPAA, region), and managed-service premium. Infrastructure as code (Terraform/Pulumi) prevents orphaned, expensive GPUs.

Cross-Connection Scenarios

Cost (2.4, 10.6); data security and residency (14); GitOps for infra (15).

Production Anti-Patterns

- Forgotten idle GPU instances (set budgets/alarms, auto-shutdown).
- Training on Spot without checkpointing.
- Storing weights only on instance disk.

Minimal Working Code

```
def gpu_cost(hourly: float, replicas: int, utilization: float = 1.0, h
ours=730):
    monthly = hourly * replicas * hours
```

```
    return monthly, monthly / max(utilization, 0.01)

def breakeven_requests(api_cost_per_req: float, monthly_gpu: float) -> int:
    return int(monthly_gpu / api_cost_per_req)

monthly, effective = gpu_cost(1.2, 2, 0.4)
print(f"${monthly:,.0f}/mo; effective at 40% util ${effective:,.0f}")
print("break-even vs API:", breakeven_requests(0.002, monthly), "requests/month")

# Terraform sketch:  resource "aws_instance" "llm" { instance_type = "g6e.xlarge
"  ... }
# Alarm: CloudWatch billing alarm + lifecycle schedule to stop dev instances nightly.
```

13.6 Performance Engineering: Caching, Routing, Gateways

Conceptual Deep Dive

Put an **LLM gateway** (LiteLLM, Portkey, Kong AI Gateway, Envoy AI Gateway) in front of providers for unified API, key management, per-team budgets, retries/fallbacks, load balancing, caching, and logging. Cache layers: exact-match, semantic cache (8, 4.6), and provider **prompt caching** (stable prefixes). Routing: cascade (try small model, escalate on low confidence), classifier-based, or latency-aware. Use request queues with priority classes, and shed load gracefully (429 with Retry-After) rather than collapse.

Cross-Connection Scenarios

Resilience patterns (4.3–4.4), cost controls (2.2), observability (14).

Production Anti-Patterns

- Direct provider SDK calls scattered across services.
- Semantic cache with a loose threshold returning wrong answers.
- No priority between interactive and batch traffic.

Minimal Working Code

```
import time

def cascade(small, large, prompt, confidence_fn, threshold=0.8):
    ans = small(prompt)
    if confidence_fn(prompt, ans) >= threshold:
        return ans, "small"
    return large(prompt), "large"

class Fallback:
    def __init__(self, providers):               # list of (name, fn)
        self.providers, self.down_until = providers, {}

    def call(self, prompt):
        for name, fn in self.providers:
            if self.down_until.get(name, 0) > time.time():
                continue
            try:
                return name, fn(prompt)
            except Exception:
                self.down_until[name] = time.time() + 30
        raise RuntimeError("all providers down")
```

13.7 Capacity Planning, Load Testing and SLOs

Conceptual Deep Dive

Define SLOs first: e.g. TTFT p95 under 1.5 s, inter-token latency p95 under 60 ms, availability 99.9%. Capacity model per replica: tokens/s throughput at your target latency (measure, don't guess), concurrency limit from KV-cache memory (free VRAM / KV bytes per request), and the prompt/output length mix. Replicas = peak tokens/s demand divided by per-replica throughput, plus headroom (N+1, 30–40% buffer for spikes). Load-test with open-loop arrival (requests arrive on a schedule regardless of completion), realistic length distributions, streaming enabled, and cold-start and failure-injection runs. Little's Law (L = lambda x W) links concurrency, arrival rate and latency. Edge cases: long-tail prompts that monopolise a replica, burst traffic after an outage (thundering herd), prefix-cache hit rates varying by time of day, and tail latency growth as utilisation approaches 100%.

Cross-Connection Scenarios

Benchmarks (10.6), autoscaling (13.4), cost (13.5), reliability (4.4).

Production Anti-Patterns

- Closed-loop load tests that hide queueing (clients wait for responses).
- Provisioning for average rather than peak.
- Running replicas above about 85% utilisation with strict latency SLOs.

Minimal Working Code

```
import math

def replicas_needed(peak_rps, mean_out_tokens, tok_per_s_per_replica, headroom=0.35, n_plus=1):
    demand = peak_rps * mean_out_tokens
    return math.ceil(demand / (tok_per_s_per_replica * (1 - headroom))) + n_plus

def max_concurrency(free_vram_gb, kv_bytes_per_token, avg_tokens_per_req):
    return int(free_vram_gb * 1e9 / (kv_bytes_per_token * avg_tokens_per_req))

print(replicas_needed(40, 350, 2500), max_concurrency(30, 131_072, 3000))
```

```
import asyncio
import random
import time

async def open_loop(send, rate_per_s: float, duration_s: float):
    """Poisson arrivals independent of completions; returns latencies."""
    lat, tasks, t_end = [], [], time.perf_counter() + duration_s

    async def one():
        t0 = time.perf_counter()
        await send()
        lat.append(time.perf_counter() - t0)

    while time.perf_counter() < t_end:
        tasks.append(asyncio.create_task(one()))
        await asyncio.sleep(random.expovariate(rate_per_s))
    await asyncio.gather(*tasks)
    return sorted(lat)
```

13.8 Reliability, Rollouts and Operations for Inference Services

Conceptual Deep Dive

Operate inference like any critical service. **Health**: liveness (process up) versus readiness (model loaded, warm-up request succeeded). **Graceful degradation**: shed load with 429 + Retry-After, reduce max_tokens, route to a smaller model, or serve cached answers. **Rolling updates** need readiness gating and maxUnavailable: 0; large weights make rollouts slow, so pre-pull images/weights. **Observability**: queue depth, running/waiting requests, KV-cache utilisation, TTFT/ITL histograms, GPU memory,

OOM and restart counts, per-model error rates. **GPU faults** (ECC errors, XID) require node draining and auto-replacement. **Runbooks** for OOM, slow replica, bad deploy and provider outage. Edge cases: a replica that is "ready" but degraded by a stuck CUDA context; autoscaler oscillation; cold replicas receiving traffic before warm-up; version skew between the router and engine API.

Cross-Connection Scenarios

Kubernetes (13.4), metrics and alerts (14.1), rollback criteria (15.5), circuit breakers (4.4).

Production Anti-Patterns

- Readiness probe that only checks the HTTP port.
- No warm-up request before taking traffic.
- Alerting on GPU utilisation instead of user-facing latency and errors.

Minimal Working Code

```
import time

class Readiness:
    def __init__(self, warm_fn):
        self.warm_fn, self.ready, self.last_ok = warm_fn, False, 0.0

    def warm_up(self):
        out = self.warm_fn()                # a real tiny generation, not just a ping
        self.ready = bool(out)
        self.last_ok = time.time()

    def check(self, max_age_s=60):
        return self.ready and (time.time() - self.last_ok) < max_age_s

r = Readiness(lambda: "ok")
r.warm_up()
print(r.check())
```

```
def admit(queue_depth: int, max_depth: int, priority: str):
    """Priority-aware load shedding: interactive survives longer than batch."""&q
uot;
    limits = {"interactive": max_depth, "batch": int(max_depth * 0.5)}
    if queue_depth >= limits[priority]:
        return {"status": 429, "headers": {"Retry-After": "5"}}
    return {"status": 200}

print(admit(60, 100, "batch"), admit(60, 100, "interactive"))
```

13.9 Multi-GPU Inference, Speculative Decoding and Disaggregated Serving

Conceptual Deep Dive

Models that exceed one GPU use **tensor parallelism** (each layer's matrices are split; two all-reduces per layer; needs NVLink-class bandwidth) or **pipeline parallelism** (layers split into stages; adds latency, helps across nodes). **Expert parallelism** spreads MoE experts. **Speculative decoding**: a small *draft* model proposes k tokens; the large *target* model verifies all k in a single forward pass, accepting the longest prefix consistent with its own distribution (via rejection sampling), so output quality is unchanged while latency falls when the acceptance rate is high. Variants: n-gram/prompt-lookup drafting (no draft model; great for extraction and editing), Medusa/EAGLE heads, and self-speculation. **Prefill/decode disaggregation** runs compute-bound prefill and memory-bound decode on separate GPU pools, transferring KV cache between them, improving both TTFT and throughput isolation. Other techniques: chunked prefill, prefix-cache-aware routing (send requests sharing a prefix to the same replica), and KV-cache offload to CPU. Edge cases: low acceptance rate making speculation slower, KV transfer cost dominating for short prompts, and TP degree not dividing the number of attention heads.

Cross-Connection Scenarios

KV cache and attention (5.4), distributed training concepts (5.10), engines (13.1–13.2), capacity planning (13.7).

Production Anti-Patterns

- Tensor parallelism across nodes connected by Ethernet.
- Enabling speculative decoding without measuring acceptance rate on real traffic.
- Round-robin routing that destroys prefix-cache locality.

Minimal Working Code

```
import random

def speculative_step(draft_probs, target_probs, rng=random):
    """One speculative-sampling decision for a drafted token.
    draft_probs/target_probs: dict token->prob at this position. Returns (token, accepted)."
;"
    tok = rng.choices(list(draft_probs), weights=list(draft_probs.values()))[0]
    accept_p = min(1.0, target_probs.get(tok, 0.0) / draft_probs[tok])
    if rng.random() < accept_p:
        return tok, True
    resid = {t: max(0.0, target_probs.get(t, 0.0) - draft_probs.get(t, 0.0)) for t in target_probs}
    z = sum(resid.values()) or 1.0
    return rng.choices(list(resid), weights=[v / z for v in resid.values()])[0], False

def expected_speedup(alpha: float, k: int, draft_cost_ratio: float) -> float:
    """alpha = per-token acceptance rate; draft_cost_ratio = draft step time / target step
 time."""
    accepted = (1 - alpha ** (k + 1)) / (1 - alpha)       # expected tokens per target pass
    return accepted / (1 + k * draft_cost_ratio)

print(round(expected_speedup(0.8, 4, 0.1), 2), round(expected_speedup(0.4, 4, 0.1), 2))
```

```
import hashlib

def prefix_aware_route(prompt_prefix: str, replicas: list[str], load: dict[str, int], max_imbalance: int
= 8):
    """Hash the shared prefix to a preferred replica; fall back if it is overloaded."
""
    h = int(hashlib.sha256(prompt_prefix.encode()).hexdigest(), 16)
    preferred = replicas[h % len(replicas)]
    least = min(replicas, key=lambda r: load[r])
    return preferred if load[preferred] - load[least] <= max_imbalance else least

print(prefix_aware_route("You are a support bot...", ["r0", "r1", "r2&
quot;], {"r0": 3, "r1": 2, "r2": 4}))
```

13.10 Edge, On-Device and Browser Deployment

Conceptual Deep Dive

Running models locally gives privacy, offline operation, zero marginal cost and low latency, at the price of capability and device variety. Runtimes: **llama.cpp / GGUF** (CPU, Apple Metal, CUDA, Vulkan), **MLX** (Apple silicon), **ONNX Runtime** and **ExecuTorch / Core ML / TensorFlow Lite** (mobile), **WebGPU/WASM** engines (in-browser), **MediaPipe** and vendor NPUs. Sizing: a 4-bit 3B model needs about 2 GB, a 4-bit 8B about 5 GB, plus KV cache; phones typically cap at a few GB and throttle thermally. Strategies: choose small task-tuned models, quantise (Q4_K_M or similar), limit context, stream tokens, run heavy work only on charger/Wi-Fi, and use **hybrid** designs (on-device for private/simple tasks, cloud for hard ones with consent). Updates: ship model files via CDN with hash verification and staged rollout. Edge cases: memory pressure kills on mobile, battery/thermal throttling, storage size of app bundles, differing quantisation behaviour across hardware, and licence terms for redistribution.

Cross-Connection Scenarios

Quantisation (12.5), SLMs (2.7), hybrid routing (13.6), artifact verification (14.5).

Production Anti-Patterns

- Shipping a model that runs only on the newest flagship device.
- No fallback when device memory is insufficient.
- Downloading model weights over HTTP without integrity checks.

Minimal Working Code

```
def fits_on_device(params_b: float, bits: int, ctx: int, kv_kb_per_tok
: float, ram_gb: float, headroom: float = 0.4) -> dict:
    weights = params_b * bits / 8
    kv = kv_kb_per_tok * 1e3 * ctx / 1e9
    need = weights + kv
    return {"need_gb": round(need, 2), "usable_gb": round(ram_gb * (1 - headroom), 2)
,
            "fits": need <= ram_gb * (1 - headroom)}

print(fits_on_device(3, 4, 4096, 90, ram_gb=6), fits_on_device(8, 4, 8192, 130, ram_gb=6))
```

```
import hashlib
import urllib.request

def download_model(url: str, dest: str, sha256: str, chunk: int = 1 << 20):
    h = hashlib.sha256()
    with urllib.request.urlopen(url) as r, open(dest, "wb") as f:
        while data := r.read(chunk):
            h.update(data)
            f.write(data)
    if h.hexdigest() != sha256:
        raise RuntimeError("model file hash mismatch; refusing to use it")
    return dest

def choose_backend(device: dict) -> str:
    if device.get("npu"):
        return "npu runtime"
    if device.get("gpu") == "apple":
        return "mlx / metal"
    return "cpu gguf (q4)"
```

13.11 Serverless GPUs, Cold Starts and Cost-Aware Scheduling

Conceptual Deep Dive

Serverless GPU platforms (Modal, Baseten, RunPod serverless, Cloud Run GPUs, SageMaker serverless-style) bill per second and scale to zero, ideal for spiky or low-volume workloads. The dominant issue is **cold start**: container pull + weight load + engine init + warm-up can take from tens of seconds to minutes. Mitigations: slim images, weights on fast network volumes or baked into snapshots, **GPU memory snapshots**, keeping min_instances of 1 for interactive paths, smaller or quantised models, lazy loading, request queues with user-visible "warming up" states, and pre-warming ahead of known traffic. Cost-aware scheduling: batch/offline jobs on spot/preemptible capacity with checkpointing, interactive on on-demand, off-peak scheduling, bin-packing small models on shared GPUs (MIG/MPS), and autoscaling policies tied to queue depth. Decision rule: below a utilisation threshold serverless wins; above it, reserved/dedicated GPUs win. Edge cases: capacity unavailability for popular GPU types, spot preemption mid-job, per-region quotas, and egress fees for model downloads.

Cross-Connection Scenarios

Kubernetes autoscaling (13.4), cloud GPUs (13.5), batch jobs (4.8), SLOs (13.7).

Production Anti-Patterns

- Scale-to-zero on a user-facing endpoint with 2-minute cold starts.
- Spot instances for long jobs without checkpoint/resume.
- Ignoring quota limits until launch day.

Minimal Working Code

```
def breakeven_utilization(serverless_per_hr: float, dedicated_per_hr:
float) -> float:
    """Fraction of time busy above which a dedicated GPU is cheaper than paying per-second
 serverless."""
    return dedicated_per_hr / serverless_per_hr

def monthly_costs(busy_hours: float, serverless_per_hr: float, dedicated_per_hr: float, hours=730):
    return {"serverless": busy_hours * serverless_per_hr, "dedicated": hours * dedica
ted_per_hr}

print(round(breakeven_utilization(4.0, 1.8), 2), monthly_costs(150, 4.0, 1.8))
```

```
import time

class WarmPool:
    """Keep `min_warm` workers ready; report whether a request hits a cold start."&qu
ot;"
    def __init__(self, min_warm: int, cold_start_s: float):
        self.min_warm, self.cold_start_s = min_warm, cold_start_s
        self.warm = min_warm
        self.busy = 0

    def acquire(self) -> float:
        if self.busy < self.warm:
            self.busy += 1
            return 0.0
        self.warm += 1                  # scale up: pay cold-start latency
        self.busy += 1
        return self.cold_start_s

    def release(self):
        self.busy -= 1

pool = WarmPool(min_warm=1, cold_start_s=45)
print([pool.acquire() for _ in range(3)])
```